Assignment - 1

Name : Prachiti Suryawanshi
PRN : 12622521 77
Class : S.Y. B Tech. CSE - CSF

Aim :

To write a C program to perform basic matrix operations such as addition, substraction, multiplication & transpose of a matrix using two-dimensional arrays.

Objectives :

- To understand the concept of 2D array in C.
- To perform arithmetic operations on matrices.
- To implement matrix addition & substraction.
- To implement matrix multiplication using nested loop.
- To find the transpose of a matrix.
- To develop programming skills using arrays.

Theory :

A matrix is a rectangular arrangement of elements organized in rows & columns. In C programming matrices are represented using 2D arrays. Matrix operations are widely used in computer graphics, image processing, machine learning, scientific computing & engineering applications

Sundaram
FOR EDUCATIONAL USE

1) Matrix Addition
Condition – Both matrices must have the same no. of rows & columns
Formula – $C[i][j] = A[i][j] + B[i][j]$

2) Matrix Subtraction
Condition – Both matrices must have the same dimensions.
Formula – $C[i][j] = A[i][j] - B[i][j]$

3) Matrix Multiplication
Condition – No. of columns of the first matrix must be equal to the no. of rows of the second matrix
Formula – $C[i][j] = \Sigma(A[i][k] \times B[k][j])$

4) Transpose of a Matrix
Condition – The transpose of a matrix is obtained by interchanging its rows & columns.
Formula – $A[i][j] = A[j][i]$

These operations are implemented using 2D arrays, nested loops & conditional statements.

FAQs:
1) Explain matrix add., sub., mult, & transpose with suitable examples.

Sundaram
FOR EDUCATIONAL USE

Ans. Matrix addition : performed by adding the corresponding elements of two matrices.

eg. $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$

$$A + B = \begin{bmatrix} 1+5 & 2+6 \\ 3+7 & 4+8 \end{bmatrix} = \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix}$$

Matrix subtraction : performed by subtracting the corresponding elements.

eg. $A - B = \begin{bmatrix} -4 & -4 \\ -4 & -4 \end{bmatrix}$

Matrix multiplication : each element of resulting matrix is obtained by multiplying rows of the first matrix with columns of second matrix.

eg. $AB = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$

Matrix Transpose : obtained by interchanging rows & columns.

eg. $A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$ $A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$

Sundaram
FOR EDUCATIONAL USE

2) Describe how 2D arrays are used to represent matrices in C programming. Explain the process of accessing & matrix elements using nested loops.

Ans: In C programming, matrices are represented using 2D arrays. 2D array stores data in rows & columns similar to a matrix.

Syntax : int matrix[3][3];
// here matrix has 3 rows & 3 columns.

Accessing elements - Elements are accessed using row & column indices.

Syntax : matrix[1][2];

Using Nested Loops - Nested loops are used because matrices have rows & columns.

Syntax :

```
for (i=0; i<rows; i++)
{
    for (j=0; j<cols; j++)
    {
        scanf ("%d", &matrix[i][j]);
    }
}
// outer loop control rows
// inner loop control columns.
```

Sundaram FOR EDUCATIONAL USE

3) Write the algorithm for performing matrix multiplication & explain why compatibility between matrices is necessary before multiplication.

Ans. S1 - START
S2 - Input rows & columns of matrix A & B
S3 - Check compatibility
S4 - Input elements of both matrices
S5 - Use 3 nested loops
- Outer for rows of A
- Middle for columns of B
- Inner for multiplication & summation
S6 - Compute :

$$C[i][j] = C[i][j] + A[i][k] \times B[k][j]$$

S7 - Display resultant
S8 - STOP

4) Discuss the applications of matrix operations in real world domains such as computer graphics, image processing, machine learning, engineering & scientific computing.

Ans Computer graphics - used for rotation, scaling & projection of objects.
- helps create 2D & 3D animations & video games.

Sundaram
FOR EDUCATIONAL USE

Image processing — images are stored as matrices of pixels.
- matrix operations are used for filterating enhancement, & edge detection.

Machine learning - data sets are represented as matrices.
- used in AI applications such as speech & face recognition.

Engineering - used in structural analysis, electrical circuits & control units.
- supports large scale numerical calculations

5) Explain the role of nested loops in implementing matrix operations in C.

Ans. Nested loops are fundamental for matrix operations because matrices contain rows & columns.
- Outer loop processes rows.
- Inner loop processes columns

Three nested loops are required
- First loop - rows of first matrix
- Second loop - columns of second matrix
- Third loop - multiplication & summation

FOR EDUCATIONAL USE

Pseudo Code

Step1 - START

Step2 - Create function to accept values for (i=0; i<m, i++)

```
{
  for(j=0; j<m; j++)
  {
    accept values
  }
}
```

Step3 - Create display function

```
for (i=0; i<m; i++)
{
  for (j=0; j<n; j++)
  {
    display matrix
  }
}
```

Step 4 - Create add func., Declare variable

```
for (i=0; i<m; i++)
{
  for (j=0; j<n; j++)
  {
    ans[i][j] = a[i][j] + b[i][j]
```

Step 5 - Create subtraction function

```
for (i=0; i<m; i++)
{
  for (j=0; j<n; j++)
```

Sundaram
FOR EDUCATIONAL USE

```
    {
      ans[i][j] = a[i][j] + b[i][j];
    }
```

Step6 - Create multiplication function

```
for (i=0; i<m; i++)
{
  for (j=0; j<n; j++)
  {
    ans[i][j] = a[i][j] + b[i][j]
  }
}
```

Step7 - Create a transpose function

```
for (i=0; i<m; i++)
{
  for (j=0; j<n; j++)
  {
    display the switched positions
  }
}
```

Step8 - Declare main func. & variables

Step9 - Accept values

Step10 - Call all functions

Step11 - Switch(ch)

case 1 : call addition

case 2 : call subtraction

case 3 : call multiplication func.

case 4 : call transpose func.

default display "wrong input"

Step12 - Display result.

Step13 - STOP

Sundaram

FOR EDUCATIONAL USE

```c
#include <stdio.h>

int main()
{
    int a[10][10], b[10][10], result[10][10];
    int rows, cols, i, j, k, choice;

    printf("Enter number of rows and columns: ");
    scanf("%d %d", &rows, &cols);

    printf("\nEnter elements of Matrix A:\n");
    for(i = 0; i < rows; i++)
    {
        for(j = 0; j < cols; j++)
        {
            scanf("%d", &a[i][j]);
        }
    }

    printf("\nEnter elements of Matrix B:\n");
    for(i = 0; i < rows; i++)
```

```c
    {
        for(j = 0; j < cols; j++)
        {
            scanf("%d", &b[i][j]);
        }
    }

    printf("\nChoose Operation:\n");
    printf("1. Addition\n");
    printf("2. Subtraction\n");
    printf("3. Multiplication\n");
    printf("4. Transpose of Matrix A\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch(choice)
    {
        case 1:
            printf("\nAddition of matrices:\n");
            for(i = 0; i < rows; i++)
            {
                for(i = 0; i < cols; i++)
```

```
            {
                result[i][j] = a[i][j] + b[i][j];
                printf("%d ", result[i][j]);
            }
            printf("\n");
        }
        break;

    case 2:
        printf("\nSubtraction of matrices:\n");
        for(i = 0; i < rows; i++)
        {
            for(j = 0; j < cols; j++)
            {
                result[i][j] = a[i][j] - b[i][j];
                printf("%d ", result[i][j]);
            }
            printf("\n");
        }
        break;
```

```
65          case 3:
66              if(rows != cols)
67              {
68                  printf("\nMultiplication is possible only for square matrices
                        .\n");
69                  break;
70              }
71
72              for(i = 0; i < rows; i++)
73              {
74                  for(j = 0; j < cols; j++)
75                  {
76                      result[i][j] = 0;
77                      for(k = 0; k < cols; k++)
78                      {
79                          result[i][j] += a[i][k] * b[k][j];
80                      }
81                  }
82              }
83
84              printf("\nMultiplication of matrices:\n");
```

```
85              for(i = 0; i < rows; i++)
86              {
87                  for(j = 0; j < cols; j++)
88                  {
89                      printf("%d ", result[i][j]);
90                  }
91                  printf("\n");
92              }
93              break;
94
95          case 4:
96              printf("\nTranspose of Matrix A:\n");
97              for(i = 0; i < cols; i++)
98              {
99                  for(j = 0; j < rows; j++)
100                 {
101                     printf("%d ", a[j][i]);
102                 }
103                 printf("\n");
104             }
105             break;
```

```
105             break;
106
107         default:
108             printf("\nInvalid choice!\n");
109     }
110
111     return 0;
112 }
```

Output

```
Enter number of rows and columns: 2 2

Enter elements of Matrix A:
1 2
3 4

Enter elements of Matrix B:
4 3
2 1

Choose Operation:
1. Addition
2. Subtraction
3. Multiplication
4. Transpose of Matrix A
Enter your choice: 1

Addition of matrices:
5 5
5 5
```